

Humanoid Pick and Place Controller

EECE5552: Assistive Robotics

Theo McArn, Ethan Milton, RM Hoque, Adrian Ramirez

December 2025

Abstract

This paper presents a hybrid control framework for humanoid pick-and-place manipulation on the Unitree G1 robot. Lower-body balance is maintained using a Model Predictive Control formulation combined with Whole Body Control to regulate center-of-mass motion and foot contact forces. Upper-body manipulation is addressed using Proximal Policy Optimization, enabling the robot to learn a grasp-and-lift behavior in simulation. A reduced-order model is employed to simplify the learning problem by locking the torso and lower body. Simulation results demonstrate successful object lifting while maintaining balance, and reveal key challenges related to friction and long-horizon stability.

1 Introduction

Humanoid robots capable of reliably performing pick-and-place tasks are a critical step toward real-world assistive robotics. Applications ranging from elder care and in-home assistance to workplace ergonomics and industrial material handling all require robots to manipulate objects while maintaining balance, safety, and robustness under uncertainty. As global demographics shift toward an aging population and physical labor continues to pose injury risks, humanoid manipulation systems that combine stability with dexterity have become an increasingly active area of research and development. (1; 2; 3)

Pick-and-place tasks on humanoids are fundamentally challenging due to the tight coupling between locomotion, balance, and manipulation. Unlike fixed-base manipulators, humanoid robots must actively regulate contact forces at the feet to maintain stability while upper body motions generate significant disturbances. These challenges are amplified when manipulating objects with nontrivial mass, friction uncertainty, or imperfect grasp geometry. As a result, effective humanoid manipulation systems often require a combination of model-based control for whole-body stability and learning-based approaches for handling contact-rich interactions.

In this project, we develop a pick-and-place control framework for the Unitree

G1 humanoid robot that explicitly separates lower body stability from upper body manipulation. The lower body is controlled using a Model Predictive Control (MPC) formulation combined with Whole Body Control (WBC) to regulate center-of-mass motion and contact forces at the feet. This approach ensures balance and physical feasibility while the robot interacts with external objects. The upper body manipulation task—grasping and lifting a box—is addressed using reinforcement learning (RL), incorporating Proximal Policy Optimization (PPO) to learn a robust grasp-and-lift policy in simulation.

To make the learning problem tractable, we introduce a reduced-order model (ROM) for the upper body that locks the torso, pelvis, and lower limbs while restricting the action space to the robot’s arms. This decoupled architecture allows each control strategy to focus on its strengths: MPC and WBC provide physically grounded stability guarantees for the lower body, while RL handles the high-dimensional, contact-rich dynamics of manipulation that are difficult to model analytically. The primary contribution of this work is not a novel control algorithm, but rather a practical, modular control pipeline that integrates established MPC, WBC, and RL techniques into a coherent humanoid pick-and-place system.

2 Related Work

Humanoid manipulation has traditionally been approached through model-based control frameworks that explicitly reason about contact forces, dynamics, and stability. One influential line of work uses Model Predictive Control (MPC) to regulate whole-body motion by optimizing future contact forces subject to dynamic and frictional constraints. Di Carlo et al. demonstrated the effectiveness of convex MPC for dynamic locomotion on the MIT Cheetah 3, showing that simplified rigid-body models can achieve highly robust control when combined with fast optimization loops (4). While originally developed for quadrupeds, these ideas have since influenced humanoid control architectures.

More recently, Li and Nguyen proposed a multi-contact MPC framework for dynamic loco-manipulation on humanoid robots, enabling simultaneous walking and object manipulation through a Single Rigid Body Dynamics (SRBD) approximation (5). Their work highlights the power of MPC in coordinating foot contact forces and center-of-mass motion during manipulation tasks. However, such approaches often involve substantial computational complexity and are most effective when locomotion and manipulation must be solved jointly.

Parallel to model-based approaches, reinforcement learning has become a popular tool for robotic manipulation, particularly in contact-rich tasks where accurate modeling is difficult. Proximal Policy Optimization (PPO) has emerged as a commonly used algorithm due to its stability and sample efficiency in continuous control domains (6). Prior work has shown that PPO-based policies can successfully learn grasping, lifting, and object reorientation behaviors when trained in physics simulation, especially when paired with carefully designed

observation spaces and reward functions (7).

However, applying reinforcement learning directly to full humanoid systems remains challenging due to the high dimensionality of the state and action spaces, as well as the difficulty of learning stable whole-body behaviors from sparse rewards. Many successful systems therefore rely on reduced-order models, symmetry assumptions, or hierarchical control structures to constrain the learning problem. Locking subsets of joints or isolating manipulation to the upper body is a common strategy to improve learning efficiency and robustness (5).

3 Lower Body Control

3.1 MPC Formulation

Our lower body MPC controller was based on a simplification of the work presented in (5). In this work, Li et al. present an MPC controller for humanoid loco-manipulation, which enables the robot to pick up objects and walk around holding these objects. However, because we were only concerned with static manipulation, we were able to remove a lot of complexity with their approach. At its core, the MPC controller used a Single Rigid Body Dynamics (SRBD) model to capture the stability and control of the humanoid. As the name suggests, this model assumes the robot to be a single rigid body which stays stable due to contact forces, linear and angular, at the feet of the humanoid. A diagram of this model, from the paper, can be seen in Figure 1. The robot state feedback, x , included the orientation of the torso, Θ , and position of the center of mass p_c , as well as their respective first time derivatives, ω and $\dot{p}_c$ respectively:

$$x = \begin{bmatrix} \Theta \\ p_c \\ \omega \\ \dot{p}_c \end{bmatrix}$$

The current state and desired state are then used to determine the desired contact forces which takes the shape:

$$u = \begin{bmatrix} F_1 \\ F_2 \\ M_1 \\ M_2 \end{bmatrix}$$

Where F_n represents the x, y and z linear reaction forces at foot n , and M_n represents the y and z reaction moments at foot n . (Note: the x moment is purposefully ignored on the assumption that the locomanipulation will not result in a lot of roll motion that would need correcting).

Using a state transition function, in the form $\dot{x} = A_c x + B_c u$, the current state, x , and current reaction forces, u , can be combined to determine the next predicted state. This step of looking ahead in time is at the heart of MPC

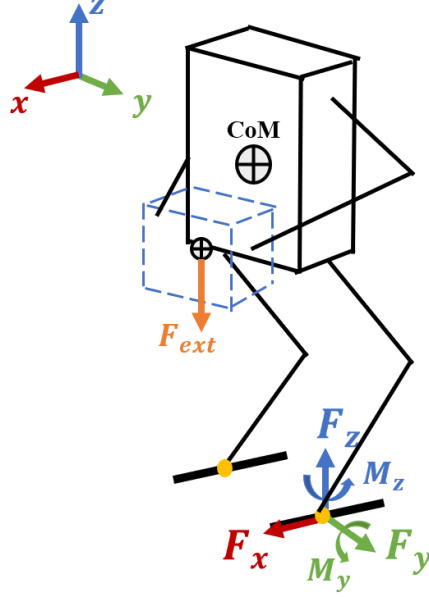


Figure 1: SRBD model for Humanoid Manipulation Problem

optimization. The idea is that for a specific horizon, in this case for 20 time-steps, we try to optimize the reaction forces, u , in a way that minimizes the distance between the desired state and the current state, which is subject to the dynamics of the state transition function. The complete formulation of this MPC optimization, and other constraints such as the friction cone, can be seen in Equation 1.

$$\min_{\mathbf{x}, \mathbf{U}} \sum_{i=0}^{k-1} (\mathbf{x}_{i+1} - \mathbf{x}_{i+1}^{ref})^T \mathbf{Q}_i (\mathbf{x}_{i+1} - \mathbf{x}_{i+1}^{ref}) + \mathbf{R}_i \|\mathbf{U}_i\| \quad (1)$$

$$\text{s. t. } \mathbf{x}[i+1] = \hat{\mathbf{A}}[i] \mathbf{x}[i] + \hat{\mathbf{B}}[i] \mathbf{U}[i], \quad (2)$$

$$\begin{aligned} -\mu F_{nz} &\leq F_{nx} \leq \mu F_{nz} \\ -\mu F_{nz} &\leq F_{ny} \leq \mu F_{nz} \end{aligned} \quad (3)$$

$$0 < F_{min} \leq F_{nz} \leq F_{max} \quad (4)$$

$$\mathbf{F}_{ext} = \begin{bmatrix} 0 & 0 & m_o g \end{bmatrix} \quad (5)$$

3.2 MPC Results

The first step of this project was to implement this MPC optimization to solve for the contact forces and verify that it was working. It was crucial to make sure

that this part works well before moving on since Whole Body Control depends on the contact forces being correct.

We verified that our implementation was working correctly by locking all joints on the G1, to make it all one single rigid body, and then externally applying the contact forces $\mathbf{u}$, that were returned from our MPC controller. Figure 2 shows the desired contact forces that are applied externally to the single rigid body. As you can see in Figures 3 and 4, this method proves successful in maintaining the desired center of mass position and the desired orientation of Unitree G1.

These results verify that the MPC algorithm is correctly predicting and correcting for disturbance by providing optimal contact forces. With this working correctly, we moved on to the task of converting these ideal reaction forces into joint torques that are realizable by the G1.

The implementation of our MPC controller can be found in our repository at `g1_mpc.py`

3.3 WBC Formulation

Whole Body Control describes the task of converting desired task space acceleration, $\ddot{x}_{des}$ into realizable acceleration in the joint space, $\ddot{q}$. Firstly, the desired task space acceleration must be extracted from the MPC controller. $\ddot{x}_{des}$ is determined by plugging the current state, x_{curr} , into the state transition function, along with the desired contact forces, u_{des} from MPC:

$$\begin{bmatrix} 0 \\ \ddot{x}_{des} \end{bmatrix} = \begin{bmatrix} \omega \\ \dot{p}_c \\ \dot{\omega} \\ \dot{p}_c \end{bmatrix} = A_c x_{curr} + B_c u_{des} \quad (6)$$

Once $\ddot{x}_{des}$ is determined, we must convert it to acceleration in the joint space, $\ddot{q}_{cmd}$. The simplest way to do this conversion is to utilize the Jacobian. The Jacobian typically maps linear velocity in the task space to angular velocity in the joint space but by differentiating this equation and solving for $\ddot{q}$ we get the following equation:

$$\ddot{q} = J^\dagger(\ddot{x} - \dot{J}\dot{q}) \quad (7)$$

where $J^\dagger$ is the pseudoinverse of J . Since our controller is running at a frequency of 50 Hz, we make the assumption that $\dot{J}\dot{q} \approx 0$. Therefore the final form for our joint space acceleration equation is:

$$\ddot{q}_{cmd} = J^\dagger \ddot{x}_{des} \quad (8)$$

where

$$J = \begin{bmatrix} J_\Theta \\ J_{p_c} \end{bmatrix} \quad \ddot{x}_{des} = \begin{bmatrix} \dot{\omega} \\ \ddot{p}_c \end{bmatrix} \quad (9)$$

Once we have $\ddot{q}_{cmd}$ we can then calculate our desired torque values, τ , for each actuator:

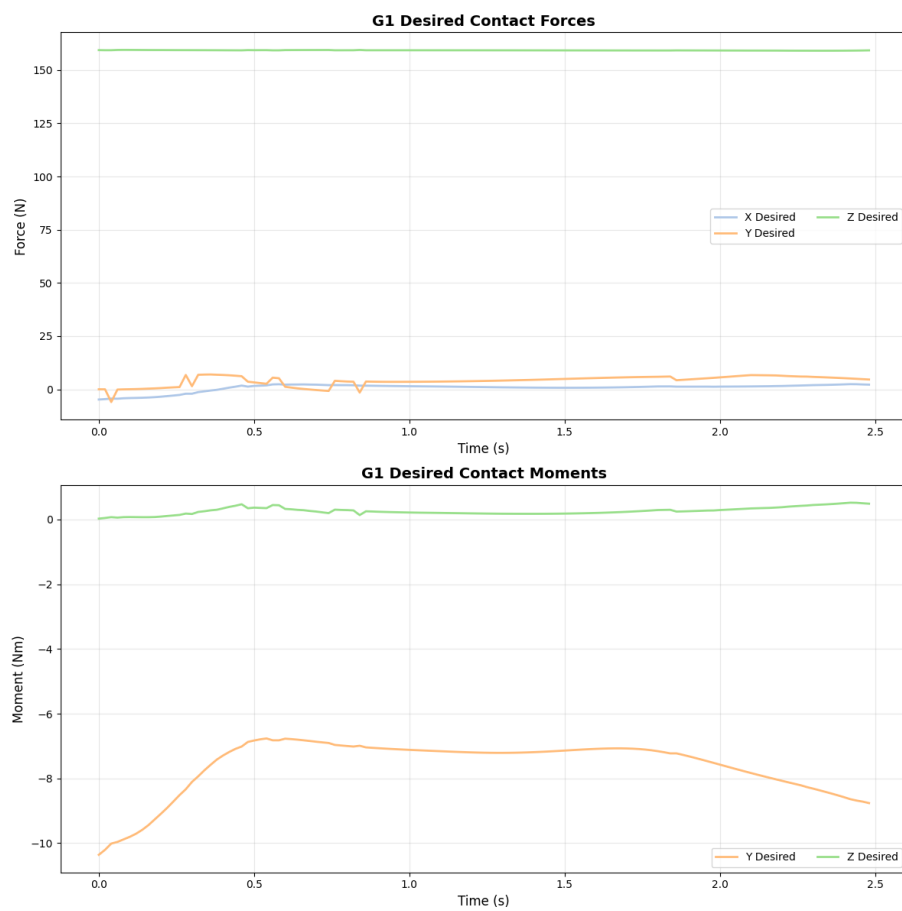


Figure 2: Contact Forces Applied Directly with SRBD

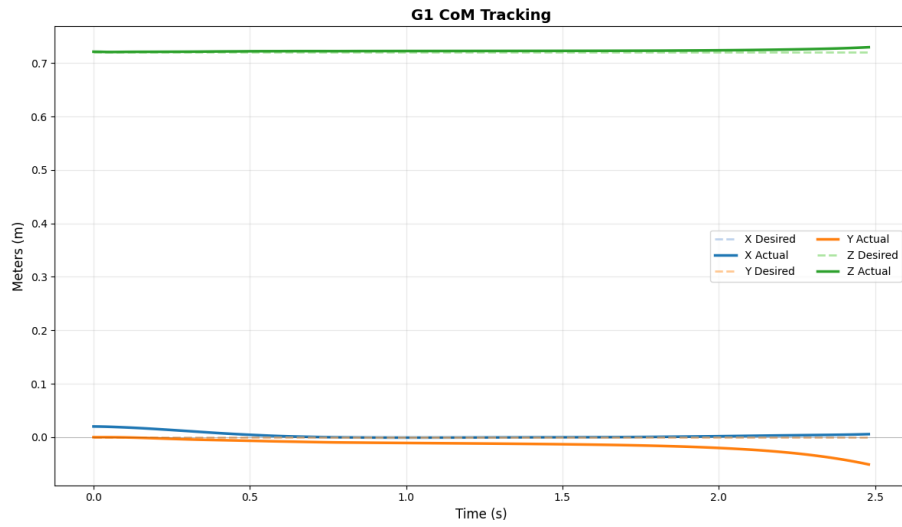


Figure 3: CoM Position - Contact Forces Applied Directly with SRBD

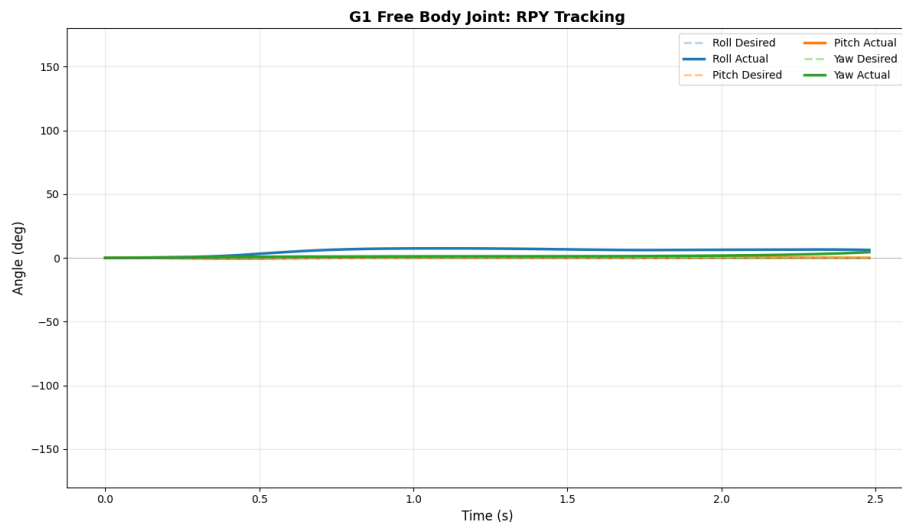


Figure 4: Orientation - Contact Forces Applied Directly with SRBD

$$\tau = M\ddot{q} + C + g - J_{con}^T u - J_{ext}^T F_{ext} \quad (10)$$

However, because $J^\dagger$ is a pseudo-inverse of J , $\ddot{q}_{cmd}$ will not directly correspond to the desired task space acceleration. As a result, we will need to perform a relaxation of u and $\ddot{q}_{cmd}$ so that they both correctly follow the dynamics in Equation 10. This relaxation can be written in the following form:

$$\min_{\Delta\ddot{q}, \Delta u} \Delta\ddot{q}^T H \Delta\ddot{q} + \Delta u^T K \Delta u \quad (11)$$

$$\begin{aligned} \text{s. t.} \quad & \mathbf{M}(\Delta\ddot{q} + \ddot{q}_{cmd}) + \mathbf{C} + \mathbf{g} \\ & J_{con}^T(\Delta u + u) - J_{ext}^T F_{ext} = \mathbf{0} \end{aligned} \quad (12)$$

$$\mathbf{u}_{min} \leq \Delta \mathbf{u} + \mathbf{u} \leq \mathbf{u}_{max} \quad (13)$$

$$\boldsymbol{\tau}_{min} \leq \boldsymbol{\tau} \leq \boldsymbol{\tau}_{max} \quad (14)$$

Where H and K are both weight matrices emphasizing the importance of keeping Δu and $\Delta\ddot{q}$ as small as possible. After performing the relaxation, the final values $(\Delta\ddot{q} + \ddot{q})$ and $(\Delta u + u)$ can be put into Equation 10 to get the final desired torque for each actuator.

3.4 WBC Results

After implementing WBC as shown above, the results were not as promising as our initial MPC results. The G1 now was only able to stand for about 2 seconds and was not able to carry nearly as much of a payload.

Figure 5 shows the desired contact forces generated from MPC. Figure 6 shows the desired task space acceleration that is determined from the MPC results. As you can see in Figures 3 and 4, this method is not successful. The true center of mass and orientation quickly begin to deviate from the desired values after around 1 second. We are not sure exactly what the issue is but we suspect there is some issue with calculating the desired task space acceleration. As you can see in Figure 6, there is a really high desired acceleration around the z-axis which intuitively does not make sense. Additionally, the desired linear acceleration in the z direction stays consistent at zero, despite the G1 falling towards the ground. This also points to this calculation not being correct.

As a result of these issues with WBC, we were unsuccessful at getting the G1 to remain standing for more than a couple seconds.

Our WBC implementation, based on the above formulation, can be found in our repository at `g1.wbc.py`

3.5 Lower Body Control Future Improvements

This method described above resulted in a lower body controller with minor success. It is clear from the render from our final results, that the G1 is correcting for the added weight of the box by leaning back, additionally, the G1 is

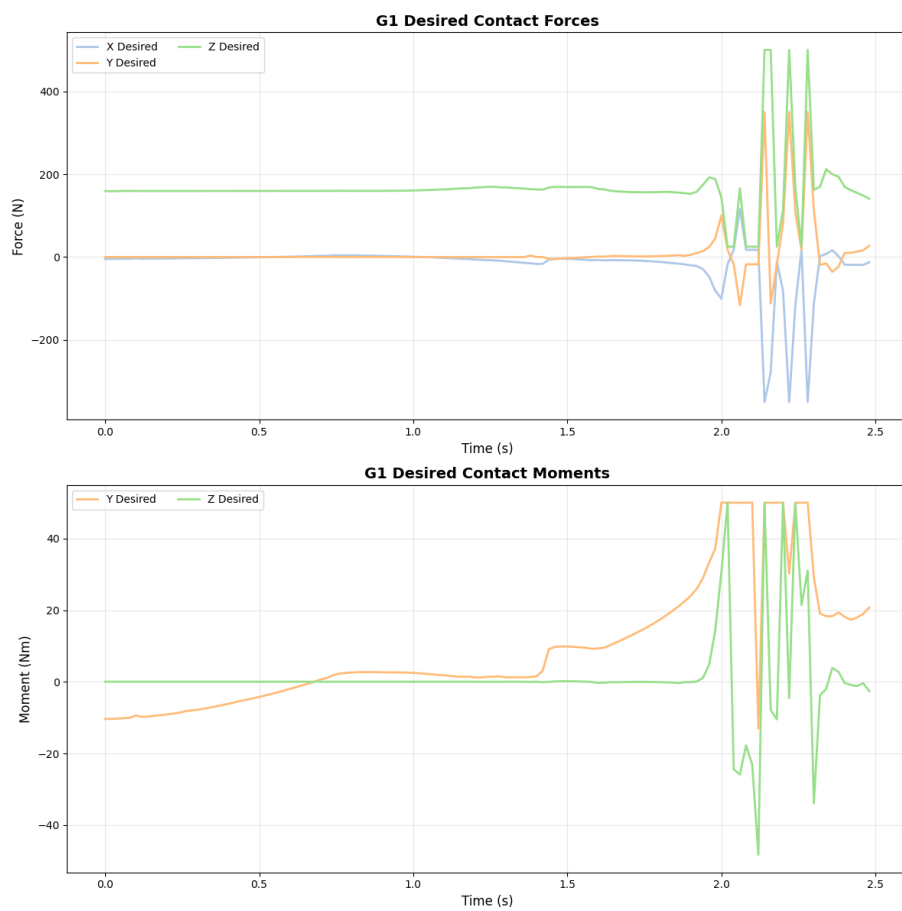


Figure 5: Desired Contact Forced with WBC

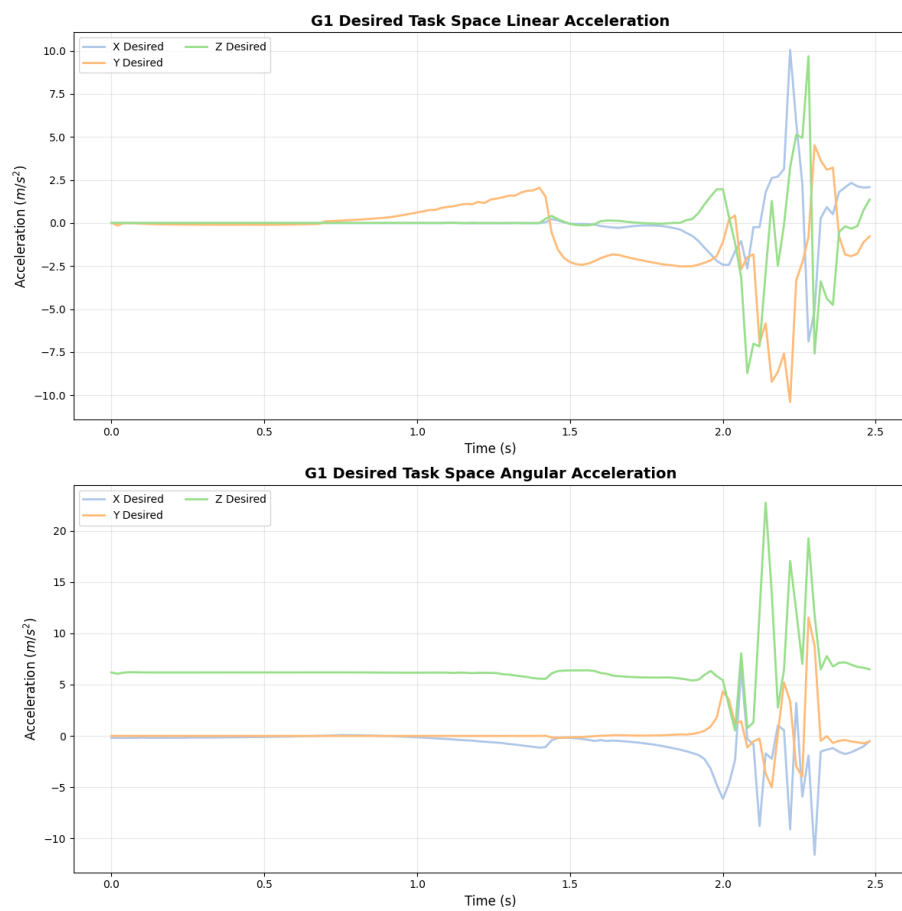


Figure 6: Calculated Desired Task Space Acceleration

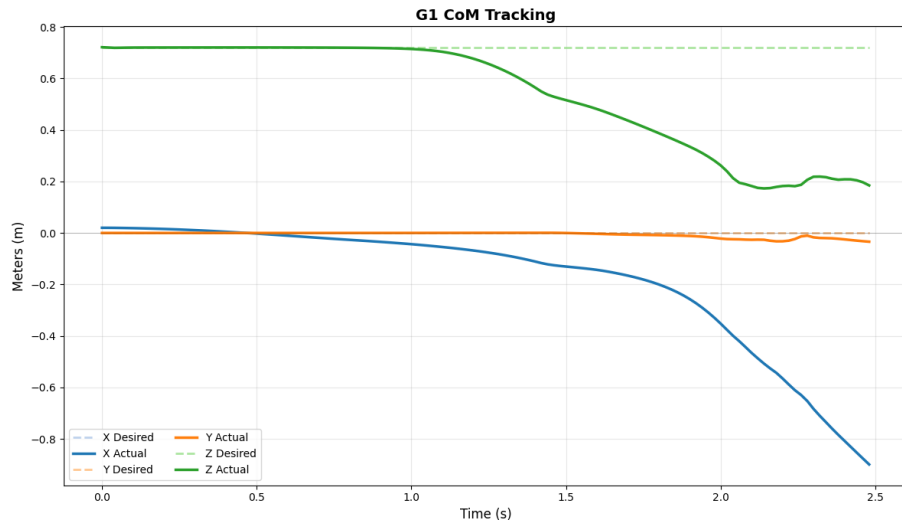


Figure 7: Center of Mass Tracking with WBC

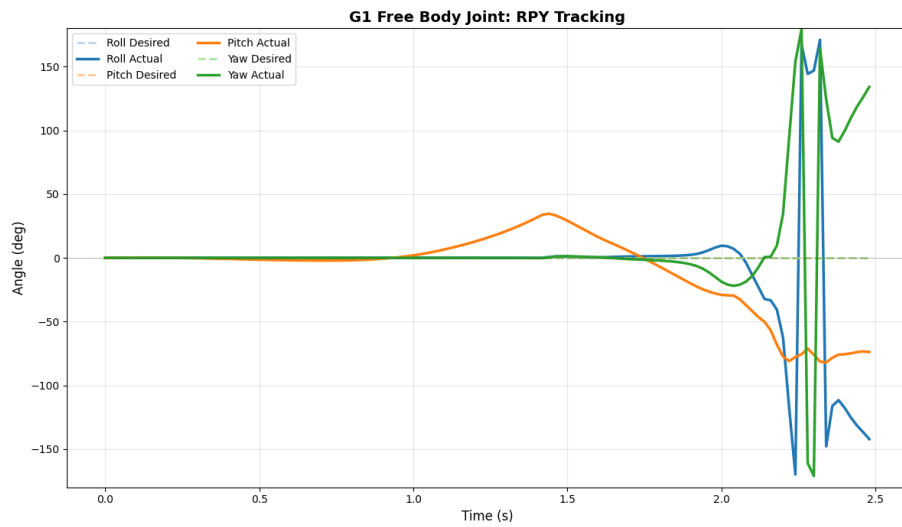


Figure 8: Orientation Tracking with WBC

correctly responding to tipping backwards by digging its heels into the ground. This shows that the underlying framework is correct there are just a few implementation errors that need correcting. As a next steps, the first thing to do is fix these errors. Additionally we think it may be beneficial to re-implement this control policy by using a more efficient optimization strategy, based on the work in (4). As of now, this policy cannot be run in real time, but by converting the optimization problem to a QP optimization, this issue could be easily fixed.

4 Upper Body Control

4.1 Upper Body Reduced Order Model

When creating an RL policy for a robot as complex as the G1, especially for a somewhat complex task such as lifting a box in a stable orientation, it is important to simplify the problem as much as possible to provide the greatest possible opportunity for successful training. To do this, we will create a reduced order model (ROM) which will serve as a more sophisticated simplification than arbitrarily removing parameters.

Since our RL sub-policy is only concerned with the arms of the robot, we wish to avoid issues such as the robot falling over or internal reactive moments affecting our policy. To do this, we can lock the torso, pelvis, and lower body of the robot, limiting our action space to the shoulders, elbows, wrists, and fingers. In Mujoco, this simply consists of locking these links to the world frame.

If we think of lifting a box only by pressing in from the sides with our hands, treating our arms similar to a large pair of grippers, we observe that the use of our fingers is not strictly necessary. Thus, by locking the G1’s fingers in an open position, we can remove fingers from our action space as well.

Additionally, we observe that with this modality of picking up the box from its sides, our arms are symmetric with respect to the center plane of the box. If we start the box aligned symmetrically with the robot (which we saw as an acceptable limitation for the scope of our project), we can mirror the right and left arm joints. This reduces our action space to 3 DOF at the shoulder, 1 DOF at the elbow, and 1 yaw DOF at the wrist. The robot, locked from the torso down and exhibiting its full symmetry, can be seen in Figure 9.

In practice, however, we found that our choice to lock or unlock the fingers had a negligible effect on performance (good or bad). Additionally, if the box became misaligned with the midplane of the robot, mirroring the joints could become a limitation rather than an improvement. When arm-symmetry was removed, bringing the action space up to 10 DOF, the training was still able to run at acceptable rates with four parallel environments, and the robot was able to learn to mirror its arms on its own. A similar pattern emerged with the physics of lifting the box.

Before we lift the box, the robot must first place its hands in the correct locations on the sides of the box. In theory, a simple linear or exponential cost function rewarding a minimization of the distance between the robot’s hands

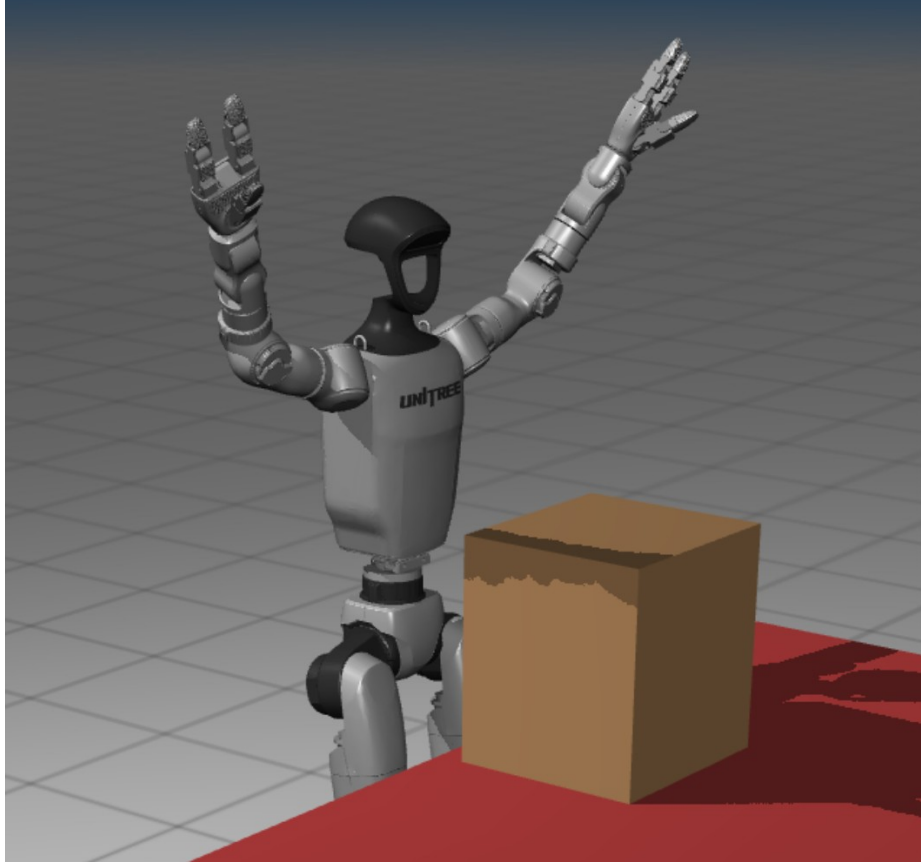


Figure 9: G1 with locked lower body and symmetric upper body

and their respective box sides would suffice. Additional rewards can be given when both hands come in contact.

Once the robot has placed its hands on the sides of the box, the robot must first ensure the box doesn't slip from its grasp. When gripping from the sides, each hand applies an inward normal force N . Friction at each contact resists the box's weight, with a maximum static friction $f_s = \mu_s N$ per side. Since there are two contacts, the total friction is $2\mu_s N$, which must balance the box's weight mg for it to remain stationary, where m is the mass of the box, and g is the acceleration due to gravity.

The horizontal force needed from each hand is thus given as:

$$N = \frac{mg}{2\mu_s} \quad (15)$$

If the robot lifts the box upward with acceleration a , the effective downward

load increases to $m(g + a)$, and the required grip then becomes:

$$N = \frac{m(g + a)}{2\mu_s} \quad (16)$$

There is also the possibility of the robot gripping with too much force and crushing the box. To avoid this, we can specify an acceptable range of forces such as 1.5-2x the required grip force.

Initially, all of the simplifications and rewards mentioned above were implemented. However, in practice we found that an overabundance of reward functions and punishments, if not very carefully managed, could actually harm the training process, as the model wouldn't know which parameters to focus on the most. Furthermore, it was extremely difficult to train the robot's hands to reach for and stay at a specified position in 3D space. Even when all other rewards were removed, the policy still seemed to resist getting closer than 10 cm from a target position, and resisted slowing down or stopping even when movement was punished. Whether this is due to bugs in our code or a more fundamental issue is, as of yet, an open question.

In practice, we found the best way to train the robot to grasp and lift the box was to remove nearly every reward except for box height, and start its hands close to its target destination on either side of the box. The results of this final set of simplifications and initial conditions are shown below.

4.2 RL Formulation

Our reinforcement learning approach uses the Stable-Baselines3 implementation of Proximal Policy Optimization (PPO) to train a policy for grasping and lifting the cardboard box. The system consists of three main components: a custom Gymnasium environment that wraps the MuJoCo physics simulator, a training script that manages the learning process, and an evaluation script for testing trained policies or checkpoints throughout the training process.

The environment (`gl_box_grasp_env_both_arms.py`) implements the standard Gymnasium interface with methods for `reset()`, `step()`, and `render()`. At each timestep, the agent receives an observation vector and outputs an action vector representing desired joint positions for the robot's arms. The environment then steps the physics simulation forward and returns the new observation, a scalar reward signal, and termination flags. It should be noted that while this process is strictly position control, the torques at the arm joints were added to the observation space, such that the robot can "see" the forces being created in its arms, and the position control can adjust accordingly.

Training is performed using parallel environments via `SubprocVecEnv`, which allows simultaneous rollout collection from multiple environment instances. This significantly accelerates training by gathering more diverse experience per wall-clock time. We use 4-8 parallel environments, depending on the abilities of each member's CPU, each running independently with different random seeds to collect experience for the PPO algorithm.

The PPO algorithm was chosen for its stability and sample efficiency in continuous control tasks. PPO constrains policy updates in such a way as to destructively large updates, making it well-suited for robotics applications where large policy changes can lead to unsafe behaviors. Our implementation uses standard PPO hyperparameters: a learning rate of $3e-4$, rollout horizon of 2048 steps, minibatch size of 64, and 10 epochs per update cycle.

Training progress is monitored using TensorBoard logging, which tracks episode rewards, policy losses, value function losses, and other training metrics. Checkpoints are saved every 5000 steps to allow resumption from failures or intentional exits. The training script supports automatic checkpoint detection and resumption, enabling long training runs to continue across interruptions. The evaluation script uses automatic checkpoint detection to find the most recent model, or the final one, if it exists. This allows us to checkup on a policy as it trains, a vital step that saved us immense amounts of time by allowing us to quit failing policies early and iterate.

4.3 RL Implementation

4.3.1 Action and Observation Space

Our final action space consists of 10 continuous values representing desired joint angles for both arms: 5 DOF per arm (3 shoulder joints for pitch/roll/yaw, 1 elbow joint, and 1 wrist yaw joint). Actions are restricted to the physical joint limits defined in the robot’s URDF file. The wrist roll and pitch joints are locked at fixed angles to keep the palms facing relatively inward toward the box, and all finger joints are frozen in an open grasping position.

The observation space is a 60-dimensional vector containing:

- Joint angles (qpos) for both arms (10 values)
- Joint velocities (qvel) for both arms (10 values)
- Motor torques for both arms (10 values)
- Box position (3 values) and quaternion orientation (4 values)
- Box linear and angular velocity (6 values)
- Averaged hand positions from finger bases (6 values total, 3 per hand)
- Relative vectors from box center to each hand (6 values)
- Target information: target height, current height, and height error (3 values)

Importantly, the observation includes both proprioceptive state (joint angles, velocities, torques) and exteroceptive information (box pose and motion), giving the agent full state information about the task.

4.3.2 Reward Function

The reward function consists of multiple weighted components that guide the learning process:

Finger Contact Reward ($w = 1.0$): Rewards individual finger contact with the box, with bonuses for multi-finger engagement. This encourages stable grasping using multiple contact points.

Palm Orientation Reward ($w = 10.0$): This reward computes the normal vector to the plane formed by the three fingertips on each hand and rewards alignment with the world Y-axis. This keeps the palms flat against the box faces rather than approaching at awkward angles. The alignment is computed as the absolute dot product between the palm normal and the Y-axis unit vector. Despite heavily weighing this reward, the robot still currently learns to use the sides of its hands, and avoids 3-finger contact with the box. This is an ongoing issue with our model and if solved could have drastic improvements on the stability of the grasp and overall performance.

Lift Height Reward ($w = 10.0$): Only active when both hands are touching the box. Provides large positive reward proportional to lift progress toward the target height of 0.82m, linearly. Heavily penalizes overshooting the target, quadratically. Includes a bonus for achieving the target height within 5cm tolerance.

Stability Reward ($w = 8.0$): Only active when the box is lifted. Rewards low box velocity, minimal XY drift from starting position, and upright orientation. Uses exponential reward shaping to create smooth gradients. Box drift and orientation are a key remaining factor in termination and poor longer term performance in our model. The robot as of yet cannot hold the box stable for more than 5-7 seconds.

Control Cost ($w = 0.1$): Small penalty on squared control inputs to encourage energy efficiency and smooth motions.

Alive Bonus ($w = 0.1$): Small constant positive reward for each timestep to encourage task persistence.

We experimented with many other reward components during development (reaching rewards, grip force rewards based on the friction calculations from our ROM, explicit movement penalties) but found that simpler was better—the agent learned more robustly with fewer, clearer objectives.

4.3.3 Termination Conditions

Episodes terminate early if any of several failure conditions occur:

- Box falls below starting height by more than 20cm
- Box moves more than 15cm in XY from starting position
- Box rotates more than 15 degrees from upright orientation (checked by comparing box frame axes to world axes)
- Robot’s hands collide with each other

- Any robot body part touches the table
- Robot torso height drops below 0.5m (indicating a fall)
- Maximum episode length of 250 steps is reached. (This termination rarely occurred as other conditions usually occurred first.)

These termination conditions were carefully tuned to give the agent enough freedom to explore while preventing obviously bad behaviors from continuing unnecessarily. However it is important to note that the reward function and termination conditions may need to be tweaked in parallel. When focussing on rewarding box stability, for example, a more lenient or less lenient tilt termination condition might be required.

4.3.4 Training Details

We trained for 1,000,000 timesteps using four (or sometimes eight) parallel environments, resulting in 250,000 episodes of experience collection (accounting for the rollout horizon). The training runs typically completed in 8-12 hours on a standard desktop with GPU acceleration.

Initial hand positions were randomized with noise (± 0.01 radians) at each episode reset to improve policy robustness. The box position had minimal randomization (kept at $x = 0.38$, $y = 0.0$, $z = 0.76$) since we accepted symmetric starting conditions as a reasonable simplification for our project scope.

We found that training stability improved significantly when we started the hands close to their target positions on either side of the box. Early training attempts with hands starting in a more neutral “arms down” position struggled to discover the grasping behavior, often getting stuck in local minima where the robot would wave its arms without making meaningful progress toward the box. In general, we were able to get the G1 to reach over the table, or to lift the box, but not both.

The final trained policy demonstrates grasping and lifting behavior, successfully picking up the box and raising it to the target height in the majority of evaluation episodes. This policy in action can be viewed [here](#).

5 Discussion / Future Work

Overall, the MPC+WBC was able to stabilize the lower body for a few seconds but was ultimately unsuccessful, due to implementation issues with Whole Body Control. The RL policy trained to control the upper body was able to learn a reliable grip acquisition when initialized near a key grasp pose. Despite these successes, the learned policy had a few consistent failure modes. The most notably occurred after 4-8 seconds of the robot holding the box, where the robot would gradually rotated and exceed the tilt termination threshold. This failure was mostly caused by a loss of friction between the g1’s hand and the box. The second failure was due the wrist yaw actuators unable to bear the



Figure 10: Robot Arms Grabbing Box

weight of the box. This would cause a funky behavior where the robot inner part of the robot hand would be the primary contact force touching the box (as seen in Figure 10).

Future work will focus in two major areas, firstly we will focus on trying to improve our current methods and identify the issues that were leading to suboptimal results. It will be easier to solve these issues while the upper and lower body controls are decoupled.

Afterwards, we want to attempt to couple the lower and upper body controls using one control policy, namely an MPC-Guided RL policy. In this architecture, MPC is used to compute an initial estimate for the desired hand and foot locations, as well as desired contact forces to satisfy the physical constraints. These estimates are then fed into the whole body RL policy to return the optimal control for each joint. This concept of MPC guided RL is generally inspired by the work in (8) which focuses only on locomotion without any manipulation. We think it would be very interesting to take the control architecture of this paper and expand it to fit our problem.

References

- [1] Grand View Research, “U.s. elder care assistive robots market size, share and trends analysis report, 2024–2033,” 2024, accessed: Oct. 24, 2025.

- [Online]. Available: <https://www.grandviewresearch.com/horizon/outlook/elder-care-assistive-robots-market/united-states>
- [2] World Health Organization, “Ageing and health,” 2022, accessed: Oct. 24, 2025. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/ageing-and-health>
 - [3] U.S. Census Bureau, “Nearly 40 percent of adults age 65 and older have at least one disability,” 2014, cB14-218. [Online]. Available: <https://www.census.gov/newsroom/archives/2014-pr/cb14-218.html>
 - [4] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, “Dynamic locomotion in the mit cheetah 3 through convex model-predictive control,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2018. [Online]. Available: https://dspace.mit.edu/bitstream/handle/1721.1/138000/convex_mpc_2fix.pdf
 - [5] J. Li and Q. Nguyen, “Multi-contact mpc for dynamic loco-manipulation on humanoid robots,” *arXiv preprint arXiv:2209.08662*, 2023. [Online]. Available: <https://arxiv.org/abs/2209.08662>
 - [6] D. Acemoglu and P. Restrepo, “Robots and jobs: Evidence from u.s. labor markets,” National Bureau of Economic Research, Tech. Rep. 30180, 2022. [Online]. Available: <https://www.nber.org/papers/w30180>
 - [7] W. Zhang, Y. Jiang, F. U. Din Farrukh, C. Zhang, and X. Xie, “A portable accelerator of proximal policy optimization for robots,” in *Proceedings of the IEEE International Conference on Integrated Circuits, Technologies and Applications (ICTA)*, Zhuhai, China, 2021, pp. 171–172.
 - [8] H. J. Lee, S. Hong, and S. Kim, “Integrating model-based footstep planning with model-free reinforcement learning for dynamic legged locomotion,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.02662>